Práctica integradora:

"El contenedor como proceso" + "Automatización con Docker"

Esta práctica se apoya en el stack : httpd + PHP que ya tienes funcionando en el PDF de teoría, y le añade la capa de procesos y automatización

Bloque A – Procesos

1. **Identifica el proceso del contenedor en el host.** Obtén el PID del proceso principal del contenedor mi-apache-php. Compáralo con ps y pstree.

```
# PID del proceso principal del contenedor
        docker inspect --format '{{.State.Pid}}' mi-apache-php
# Ejemplo: 12345

# Ver el proceso en el host
        ps -p 12345 -o pid,ppid,cmd
#   PID  PPID CMD
# 12345     1 httpd -DFOREGROUND

# Árbol de procesos: ver el contenedor dentro de la jerarquía del host
        pstree -p 12345
```

Un contenedor es, en el fondo, un proceso del host aislado con *namespaces* y limitado con *cgroups*. El PID que devuelve inspect es el PID real en el host.

2. **Estados y ciclo de vida.** Arranca, para y destruye el contenedor. Documenta los estados (Up, Exited) y el PID en cada fase.

```
# Estado actual
        docker ps --filter name=mi-apache-php --format 'table {{.Names}}\t{{.Status}}\t{{.State}}'
# NAMES          STATUS         STATE
# mi-apache-php    Up 10 minutes   running

# Parar (SIGTERM)
        docker stop mi-apache-php

# Ver estado parado
docker ps -a --filter name=mi-apache-php --format 'table {{.Names}}\t{{.Status}}\t{{.State}}'
# mi-apache-php    Exited (0) 5 seconds ago   exited

# Arrancar de nuevo
        docker start mi-apache-php
```

Documentar: running → exited (0) → running. El PID cambia en cada arranque.

3. **Proceso vs. hilo vs. trabajo.** Dentro del contenedor, lanza un sleep 300 & (trabajo en background) y comprueba con jobs. En el host, compara el proceso httpd con sus hilos.

```
        docker exec -it mi-apache-php sh
# Dentro del contenedor:
        sleep 300 &
        jobs
        # [1]+  Running                 sleep 300 &

        ps
        # PID   USER     TIME  COMMAND
        #     1 root      0:00 httpd -DFOREGROUND
        #    10 root      0:00 httpd -DFOREGROUND
        #   ...
        #   123 root      0:00 sleep 300
```

En el host (hilos del proceso httpd):

```
# Ver hilos del proceso httpd del contenedor
        ps -T -p $(docker inspect --format '{{.State.Pid}}' mi-apache-php)
        #   PID  SPID TTY          TIME CMD
        # 12345 12345 ?        00:00:00 httpd
        # 12345 12346 ?        00:00:00 httpd
        # 12345 12347 ?        00:00:00 httpd
```

Diferencias:

- **Proceso:** httpd -DFOREGROUND (PID 1 dentro del contenedor).
- **Hilo:** cada SPID de ps -T es un hilo del mismo proceso.
- **Trabajo (job):** sleep 300 & es un trabajo en background del shell, gestionado con jobs, fg, bg

4. **Señales.** Para el contenedor con docker stop (SIGTERM) y con docker kill (SIGKILL). Documenta la diferencia en el estado final.

```
# SIGTERM (docker stop): permite al proceso terminar limpiamente
        docker stop mi-apache-php
        docker ps -a --filter name=mi-apache-php --format '{{.Status}}'
# Exited (0) ...

# SIGKILL (docker kill): terminación forzosa
        docker start mi-apache-php
        docker kill mi-apache-php
        docker ps -a --filter name=mi-apache-php --format '{{.Status}}'
        # Exited (137) ...
```

Diferencia: Exited (0) = terminación limpia (SIGTERM). Exited (137) = 128 + 9 (SIGKILL), terminación forzosa.

5. **/proc como registro.** Accede a /proc/<PID>/status, /proc/<PID>/cmdline y /proc/<PID>/fd/ del proceso del contenedor.

```
PID=$(docker inspect --format '{{.State.Pid}}' mi-apache-php)

# Estado del proceso
cat /proc/$PID/status | head -20

# Línea de comandos
tr '\0' ' ' < /proc/$PID/cmdline; echo
# httpd -DFOREGROUND

# Descriptores de fichero abiertos
ls -l /proc/$PID/fd/
```

/proc/[PID]/ es el "expediente" del proceso en el sistema de archivos virtual. Docker no lo elimina: el proceso del contenedor sigue teniendo su entrada en /proc del host.

6. **Seguridad.** Simula un contenedor "no identificado" (docker run -d --name sospechoso alpine sleep 3600) y detéctalo con docker ps -a y ps aux | grep sleep. Elimínalo.

```
docker run -d --name sospechoso alpine sleep 3600

# Detectar contenedores no esperados
docker ps -a --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
# NAMES          IMAGE              STATUS
# mi-apache-php    stack-httpd-php-web   Up 5 minutes
# sospechoso       alpine            Up 10 seconds   ← ¡no esperado!

# Detectar el proceso en el host
ps aux | grep -i sleep | grep -v grep
    # root  12345 ... sleep 3600

# Eliminar
docker rm -f sospechoso
```

Medidas de seguridad documentadas:

- Revisar periódicamente docker ps -a y comparar con la lista de servicios esperados.
- Revisar ps aux buscando procesos no habituales.
- Usar docker events para monitorizar creación de contenedores en tiempo real.
- Restringir quién puede ejecutar docker (grupo docker = root efectivo).

Bloque B – Automatización

1. **Script de automatización.** Crea ~/bin/check_stack.sh que:

   - Compruebe si el contenedor mi-apache-php está corriendo.
   - Si no lo está, ejecute docker compose up -d.
   - Registre la acción en ~/logs/check_stack.log.

```
#!/bin/bash
# check_stack.sh - Verifica y arranca el stack httpd+php si es necesario
# Autor: alumno ASIR
# Fecha: $(date +%Y-%m-%d)

LOG=~/logs/check_stack.log
STACK_DIR=~/stack-httpd-php
CONTAINER=mi-apache-php

# Crear usuario webadmin si no existe (RA3 e)
if ! id -u webadmin >/dev/null 2>&1; then
   sudo useradd -m -s /bin/bash webadmin
   echo "$(date '+%F %T') [INFO] Usuario webadmin creado" >> "$LOG"
fi

# Comprobar si el contenedor está corriendo
if docker ps --filter "name=$CONTAINER" --filter "status=running" | grep -q "$CONTAINER";
then
   echo "$(date '+%F %T') [OK] $CONTAINER en ejecución" >> "$LOG"
else
   echo "$(date '+%F %T') [WARN] $CONTAINER caído. Arrancando stack..." >> "$LOG"
   cd "$STACK_DIR" && docker compose up -d >> "$LOG" 2>&1
   if [ $? -eq 0 ]; then
      echo "$(date '+%F %T') [OK] Stack arrancado correctamente" >> "$LOG"
   else
      echo "$(date '+%F %T') [ERROR] Fallo al arrancar el stack" >> "$LOG"
   fi
fi
```

2. **Planificación con cron.** Programa el script cada 5 minutos con crontab -e (usuario sin privilegios).

```
crontab -e
```

y añadir:

```
# Verificar el stack httpd+php cada 5 minutos
*/5 * * * * /home/alumno/bin/check_stack.sh
```

Verificar con:

```
crontab -l
```

3. **Tarea puntual con at.** Programa una tarea que se ejecute en 2 minutos y haga docker compose down (simulando un mantenimiento).

```
# Programar un mantenimiento en 2 minutos
echo "cd ~/stack-httpd-php && docker compose down" | at now + 2 minutes

# Ver tareas programadas
        atq
        # 3   Mon Oct 25 12:30:00 2026 a alumno

# Ver el contenido de la tarea
        at -c 3
```

4. **Documentación.** Haz una pequeña prueba de los siguientes puntos dentro de un contendor de ubuntu, que no hayas utilizando antes :

- crontab -l,
- systemctl list-timers
- Contenido de logs del sistema.